Essentials of Database Management System (DBMS)

"A database is the organized memory of an organization."

Traditional File System

- A file-based system is a method of storing and managing data in individual files.
- These files are organized in directories or folders on a computer's storage device.

⚠ The Fatal Weakness of File System

Data Redundancy Data Inconsistency Poor Data Security Difficult Data Access

"These weaknesses make file systems inefficient, leading to the need for DBMS."

What is DBMS?

A Database Management System (DBMS) is software used to manage data from a database.

It acts as an interface between the database and end users or applications, ensuring data is consistently organized , easily accessible, and secure.

FEATURES	FILE SYSTEM	DBMS
➤ Redundancy	➤ High	➤ Low
➤ Consistency	➤ Poor	➤ High
➤ Security	➤ Weak	➤ Strong
➤ Data Access	➤ Difficult	➤ Easy
➤ Integrity	➤ Not Maintained	➤ Maintained

"DBMS overcomes limitations of traditional file systems."

Features	Relational (SQL)	NoSQL (Non-Relational)
Data Structure	Structured (Table-based)	Unstructured (Document, Key-Value)
Schema	Rigid (Defined upfront)	Flexible (Dynamic)
Query Language	SQL (Standardized)	Varies (UnQL, JSON, API-based)
Scaling	Vertical (Add power to server)	Horizontal (Add more servers)
Relationships	Supported via JOINS	Not heavily supported (often denormalized)
Transactions	ACID (Strong consistency)	BASE (Eventual consistency)
Best for	Complex queries, financial systems	High throughput, massive data, agility

KEYS IN DATABASE :

Keys in DBMS make sure of data integrity, uniqueness, and the quick retrieval of information. Key is attribute in table

TYPES OF KEYS:

Candidate Key

A candidate key refers to a group of attributes capable of uniquely identifying a record within a table. Among these, one is selected to serve as the primary key.

Ex: For student possible attributes for candidate key could be Student ID , Student Roll No.

Primary Key

A primary key is a key which uniquely identifies each record in a table. It ensures that each tuple or record can be uniquely identified within the table.

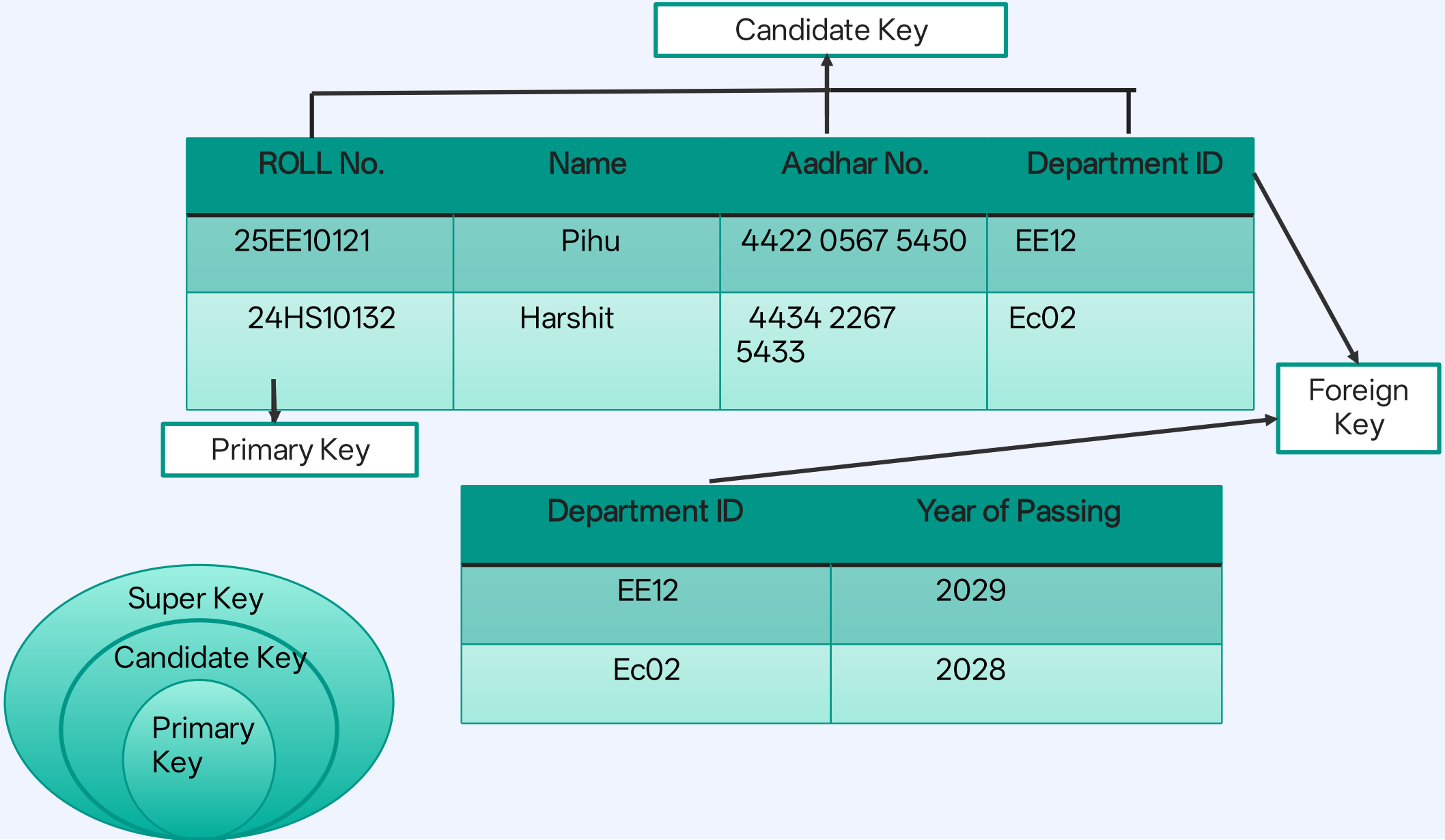
It is always **Unique+ Not null**

Foreign Key

A foreign key is a field in a table that refers to the primary key in another table. It establishes a relationship between two tables.

Super Key

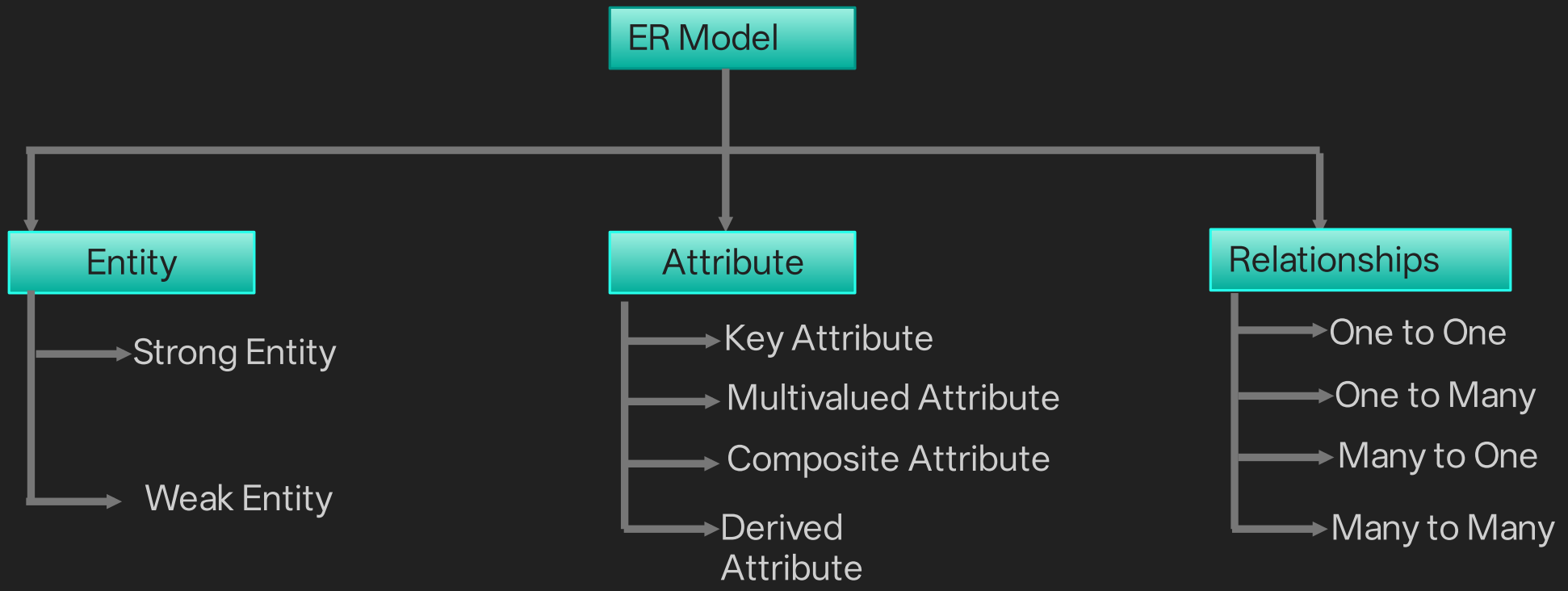
A **Super Key** in DBMS is a group of one or more attributes in a table that can uniquely identify every row in that table. It ensures no two rows have the same combination of values for those attributes.


Candidate Key
ROLL No.
Name
Aadhar No.
Department ID
25EE10121
Pihu
4422 0567 5450
EE12
24HS10132
Harshit
4434 2267 5433
Ec02
Primary Key
Foreign Key
Department ID
Year of Passing
EE12
2029
Ec02
2028
Super Key
Candidate Key
Primary Key

ER Model:

The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.

- **Entity:** An object that is stored as data. Ex : Name etc.
- **Attribute:** Properties that describe an entity. Ex: Roll no. , Department ID , Aadhar No. etc.
- **Relationship:** A connection between entities. Ex: Student belonging to specific department .

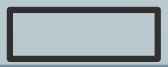


ER Diagrams in DBMS :

ER diagrams represent the E-R model in a database, making them easy to convert into relations.

Symbols used in ER Model :

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Line		Attributes to Entities and Entities sets with other Relationship types .
Diamond		Relationships among Entities
Double Ellipse		Multi-valued Attributes
Double Rectangle		Weak Entity

Referential Integrity

- Referential Integrity ensures that a foreign key must match an existing primary key ,
- Maintains **valid relationships between tables**

For Ex : In the previous table , Every Department ID in **Student Table** must exist in **Department Table**

- Pihu → EE12 ✔ (exists in Department table)
- Harshit → Ec02 ✔ (exists in Department table)

ROLL No.	Name	Aadhar No.	Department ID
25EE10121	Pihu	4422 0567 5450	EE12
24HS10132	Harshit	4434 2267 5433	Ec02

Department ID	Year of Passing
EE12	2029
Ec02	2028

Referential Integrity **keeps data consistent and prevents invalid relationships between tables**

Types of relationship in DBMS:

There are 4 types of relationship based on degree:

One to One (1-1)

One record in Table A is linked to only one record in Table B and vice versa

Many to One (N-1)

Many records in Table A relate to one record in Table B

One to Many (1-N)

One record in Table A can be linked to many records in Table B

Many to Many (N-N)

Many records in Table A relate to many records in Table B

Unvisited nodes get UCT = infinity — guaranteeing they are explored before revisiting known nodes.

EXAMPLES :

M:N (Many-to-Many)

1:1 (One-to-One)

1:N (One-to-Many)

M:1 (Many-to-One)

Weak Entity Set & Total Relationship

Weak Entity Set

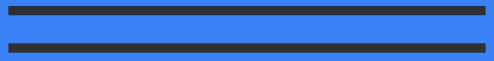
Represented by:

A **Weak Entity Set** is an entity that **cannot be uniquely identified by its own attributes** and depends on another (strong) entity.

- Does **not have a primary key**
- Has a **partial key (discriminator)**
- Depends on a **Strong Entity**
- Uses **identifying relationship**

Total Relationship

Represented by :



A **Total Relationship** means **every entity must participate** in the relationship.

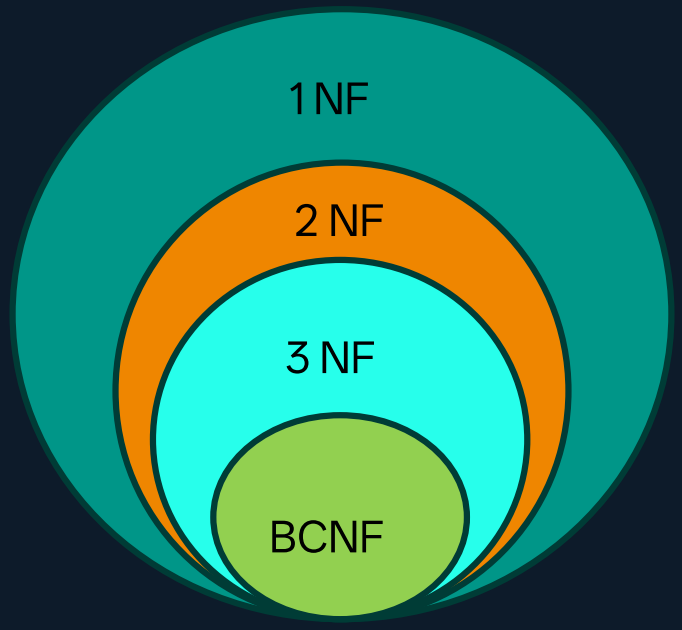
Total relationship ensures **data completeness and integrity**

Normalization and its types

Normalization is a process in which we organize data to reduce redundancy(duplicacy) and improve data consistency. It involves dividing a database into two or more tables

Benefits of using Normal Forms:

- Reduce duplicate data and wasted storage.
- Prevent insert, update, and delete anomalies.
- Improve data consistency and integrity.
- Make the schema easier to maintain and evolve.



Types of Normalization:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form(BCNF)

First Normal Form (1NF): Eliminating Duplicate Records

A table is in 1 NF if all columns contain atomic values (i.e., indivisible values).

Roll_no	Student_Name	Subjects	Dept
1	Priya	pow,bem	CSE
2	Rahul	ec,et	EC
3	Anita	lanc,bem	HS

Roll_no	Student_Name	Subjects	Dept
1	Priya	pow	CSE
1	Priya	bem	CSE
2	Rahul	ec	EC
2	Rahul	et	EC
3	Anita	lanc	HS
3	Anita	bem	HS

Each attribute is atomic → **1NF satisfied.**
→

Second Normal Form (2NF): Eliminating Partial Dependency

A relation is in 2 NF if it satisfies the conditions of 1NF and additionally. No partial dependency exists, meaning every non-prime attribute (non-key attribute) must depend on the entire primary key, not just a part of it.

It creates separate table for partial dependency.

Roll_no	Student_Name	Subjects	Dept
1	Priya	pow	CSE
1	Priya	bem	CSE
2	Rahul	ec	EC
2	Rahul	et	EC
3	Anita	lanc	HS
3	Anita	bem	HS

Roll_no	Student_Name	Dept
1	Priya	CSE
2	Rahul	EC
3	Anita	HS

Roll_no	Subjects
1	pow
1	bem
2	ec
2	et
3	lanc
3	bem

Third Normal Form (3NF): Eliminating Transitive Dependency

A relation is in 3NF if it satisfies 2NF and additionally , there is no transitive dependency for non-prime attributes then table is in the form of 3NF

Roll_no	Student_Name	Fee	Dept
1	Priya	1 lakh	CSE
2	Pari	90,000	HS
3	Rahul	98000	EC

Roll_no	Student_Name	Dept
1	Priya	CSE
2	Pari	HS
3	Rahul	EC

Fee	Dept
1 lakh	CSE
90,000	HS
98000	EC

Boyce-Codd Normal Form (BCNF):

It is stricter version of 3NF where for every non-trivial functional dependency (X → Y), X must be super key (a unique identifier for a record in the table).

Roll_no	Student_Name	Dept_ID	Dept
1	Priya	HS21	HS
2	Pari	EC22	EC
3	Payal	AG52	AG
4	Rahul	EC22	EC

Roll_no	Student_Name
1	Priya
2	Pari
3	Payal
4	Rahul

Dept_ID	Dept
HS21	HS
EC22	EC
AG52	AG

SQL (Structured Query Language)

SQL is a standard language for accessing and manipulating databases.

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987

Uses of SQL:

- Insert, update, delete data
- Retrieve data (queries)
- Manage database structure

"Data is a precious thing and will last longer than the systems themselves."

Basic SQL Queries

- CREATE DATABASE → creates a new database
- USE → selects the database to work on
- CREATE TABLE → creates a new table
- PRIMARY KEY → uniquely identifies each row
- SELECT → retrieves data from table
- DISTINCT → removes duplicate values
- LIMIT → restricts number of rows returned
- WHERE → filters records based on condition
- AND/OR/NOT → combines multiple conditions
- ORDER BY → sorts data (ASC/DESC)
- AVG/SUM/COUNT/MAX/MIN → perform calculations on data
- GROUP BY → groups rows with same values
- HAVING → filters grouped data
- LIKE → searches using patterns (wildcards)
- INNER JOIN → shows only matching records
- LEFT JOIN → shows all left table records
- RIGHT JOIN → shows all right table records

```sql
CREATE DATABASE college;
USE college;
CREATE TABLE students (
    Roll_no INT PRIMARY KEY,
    Student_Name VARCHAR(50),
    Age INT,
    Dept_ID VARCHAR(10) UNIQUE NOT NULL
);
CREATE TABLE department (
    Dept_ID VARCHAR(10) PRIMARY KEY,
    Dept_Name VARCHAR(50),
    HOD VARCHAR(50),
    Location VARCHAR(50)
);
ALTER TABLE students
ADD CONSTRAINT fk_dept
FOREIGN KEY (Dept_ID)
REFERENCES department(Dept_ID);
```

Basic SQL Queries

```
INSERT INTO department VALUES
('CSE01', 'CSE', 'Shaalu', 'Block A'),
('ECE02', 'ECE', 'Ved', 'Block B'),
('ME12', 'ME', 'Siddhi', 'Block C'),
('CE99', 'Civil', 'Ram', 'Block D');

INSERT INTO students VALUES
(1, 'Priya', 20, 'CSE01'),
(2, 'Rahul', 21, 'ECE02'),
(3, 'Anita', 22, 'CSE01'),
(4, 'Aman', 20, 'AG12');
```

```
9 •  SELECT * FROM students;
```

Roll_no	Student_Name	Age	Dept_ID
1	Priya	20	CSE01
2	Rahul	21	ECE02
3	Anita	22	CSE01
4	Aman	20	AG12

```
SELECT * FROM department;
```

Dept_ID	Dept_Name	HOD	Location
CE99	Civil	Ram	Block D
CSE01	CSE	Shaalu	Block A
ECE02	ECE	Ved	Block B
ME12	ME	Siddhi	Block C

```
SELECT * FROM students WHERE Age > 20;
```

Roll_no	Student_Name	Age	Dept_ID
2	Rahul	21	ECE02
3	Anita	22	CSE01

```
SELECT * FROM students ORDER BY Age ASC;
```

Roll_no	Student_Name	Age	Dept_ID
1	Priya	20	CSE01
4	Aman	20	AG12
2	Rahul	21	ECE02
3	Anita	22	CSE01

```
SELECT Dept_ID, COUNT(*)
FROM students
GROUP BY Dept_ID
HAVING COUNT(*) > 1;
```

Dept_ID	COUNT(*)
CSE01	2

```
SELECT COUNT(*) FROM students;
```

COUNT(*)
4

```
SELECT * FROM students WHERE Student_Name LIKE 'P%';
```

Roll_no	Student_Name	Age	Dept_ID
1	Priya	20	CSE01

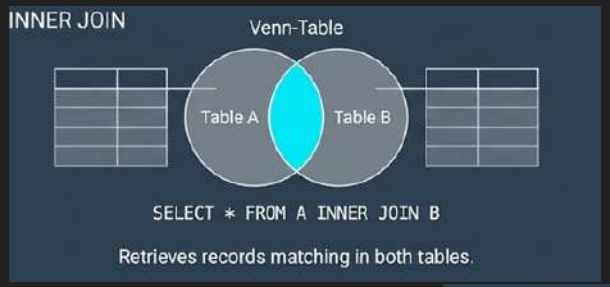
Basic SQL Queries

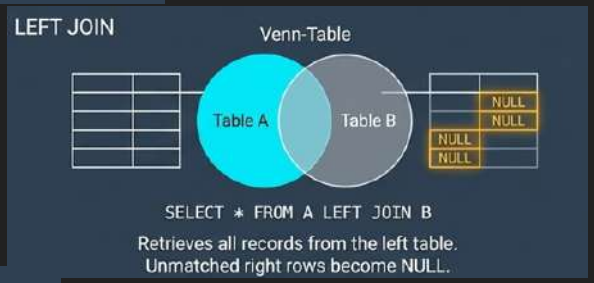
```sql
SELECT students.Student_Name, department.Dept_Name
FROM students
INNER JOIN department
ON students.Dept_ID = department.Dept_ID;
```

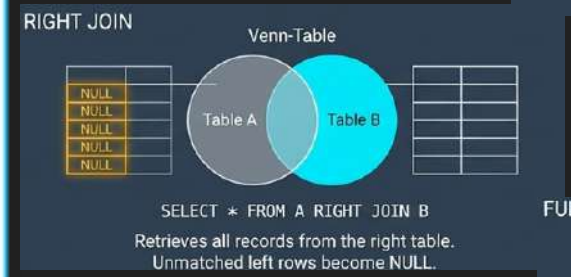
Student_Name	Dept_Name
Priya	CSE
Rahul	ECE
Anita	CSE

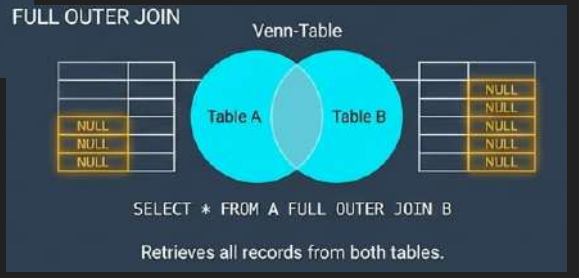
```sql
SELECT students.Student_Name, department.Dept_Name
FROM students
LEFT JOIN department
ON students.Dept_ID = department.Dept_ID;
```

Student_Name	Dept_Name
Priya	CSE
Rahul	ECE
Anita	CSE
Aman	NULL









CONCLUSION

- DBMS provides **efficient, secure, and structured data management**
- Overcomes limitations of traditional **file systems**
- **ER Diagrams** help in designing databases clearly
- **Normalization** reduces redundancy and improves efficiency.
- **SQL** enables easy data retrieval and manipulation.